Speedhawk Architecture and Service Connection Guide

A detailed breakdown of how the Speedhawk project should be separated, how each service connects, what each folder owns, and how the full audit flow works end to end.

1. The Three Main Apps / Services

Speedhawk should be separated into three main applications that talk through two shared infrastructure services. This is the cleanest way to keep the project scalable, easier to maintain, and closer to how real production systems are built.

Service	Main Role	Folder	Deploy Target
Frontend	User-facing product UI	client/	Vercel
Backend API	Traffic controller and application logic	server/	Render or Railway
Worker	Heavy background processing	worker/	Render or Railway

Frontend

This is the user-facing application.

Responsibilities:

- Landing page
- Login / signup UI
- Submit URL form
- Dashboard
- Results and history pages
- Charts and visualizations

Tech:

- React
- Tailwind CSS
- Recharts or a similar charting library

Folder: **client/**

Deploy to: **Vercel**

Backend API

This is the main application server.

Responsibilities:

- Authentication
- User management

- Validate submitted URLs
- Create audit records in PostgreSQL
- Enqueue audit jobs into Redis / BullMQ
- Return audit history and results to the frontend

Tech:

- Node.js
- Express
- PostgreSQL
- Redis client
- BullMQ producer

Folder: **server/**

Deploy to: **Render or Railway**

> Think of this as the traffic controller. It should **not** run the heavy Lighthouse audit itself.

Worker

This is the heavy-processing service.

Responsibilities:

- Listen to queue jobs
- Run Puppeteer + Lighthouse
- Parse results
- Generate suggestions
- Save metrics into PostgreSQL
- Mark the audit complete or failed

Tech:

- Node.js
- BullMQ consumer
- Puppeteer
- Lighthouse
- PostgreSQL

Folder: **worker/**

Deploy to: **Render or Railway**

> This should be separate because Lighthouse and Puppeteer are heavier and can slow or crash the API if everything is combined.

2. Why Separate Them This Way

Each piece has a different job, and separating them keeps responsibilities clear.

- **Frontend:** presentation layer
- **API:** request / response layer, business logic, data access, queue creation
- **Worker:** background processing, CPU- and memory-heavy audit jobs

That structure is much closer to how real systems are built.

3. Recommended Project Structure

```
speedhawk/
  client/
  server/
  worker/
  shared/
  docs/
  README.md
```

What each folder does

client/ - React frontend

```
client/
  src/
    components/
    pages/
    hooks/
    services/
```

server/ - Express API

```
server/
  src/
    routes/
    controllers/
    middleware/
    services/
    db/
    utils/
```

worker/ - Background job processing

```
worker/
  src/
    jobs/
    processors/
    services/
    utils/
```

shared/ - Optional but useful for shared constants and types

```
shared/
  constants/
  types/
  validators/
```

This is helpful when both the frontend and backend use the same audit status values, validation rules, or TypeScript types.

4. How the Flow Works Across Services

```
Frontend
  -> API
  -> API stores pending audit in Postgres
  -> API pushes job to Redis queue
  -> Worker consumes job
  -> Worker runs Lighthouse audit
  -> Worker saves results in Postgres
  -> Frontend fetches results from API
```

This is the clean high-level lifecycle of a Speedhawk audit.

5. What Should Not Be Mixed Together

- Do not put worker logic inside the frontend.
- Do not put Lighthouse execution directly inside API routes.

Running Lighthouse directly inside an API route is a bad idea because it creates long response times, blocks the server, makes scaling harder, and puts a heavier Puppeteer workload on the API.

Bad pattern:

```
POST /audit -> run Lighthouse immediately -> wait 20 seconds
```

Better pattern:

```
POST /audit -> create job -> return job or audit id quickly
```

6. Best Separation by Ownership for a Team

If you have 4 people

- **Person 1 - Frontend:** client/, dashboard UI, landing page, charts
- **Person 2 - API / backend:** server/, auth, routes, database integration
- **Person 3 - Worker / infra:** worker/, BullMQ, Redis, Lighthouse, Puppeteer
- **Person 4 - Full-stack / integration:** connect all services, testing, deployment, bug fixing, shared code and docs

If you have 3 people

- **Person 1:** frontend
- **Person 2:** backend API and database
- **Person 3:** worker, infrastructure, and deployment

With three people, everyone should still help on integration.

7. Monorepo vs Separate Repositories

For this team project, one monorepo is the better choice.

```
speedhawk/
  client/
```

```
server/
worker/
```

- Easier collaboration
- Easier onboarding
- One README
- One shared issue board
- Simpler for students

Separate repositories only make sense if the project becomes much larger later. For now, keep separate services, but one repo.

8. Shared Infrastructure

Postgres and Redis are shared services rather than separate codebases.

- **Postgres** is used by the API and the worker.
- **Redis** is used by the API to push jobs and by the worker to consume jobs.

```
client  -> server
server  -> postgres
server  -> redis
worker  -> redis
worker  -> postgres
```

9. How They Should All Connect

Think of Speedhawk as three apps talking through two shared services.

```
Frontend (client)
      |
      v
Backend API (server)
      | \
      |  \
      v   v
 Postgres  Redis Queue
             |
             v
        Worker service
             |
             v
          Postgres
```

- Frontend talks only to the backend
- Backend talks to Postgres and Redis
- Worker talks to Redis and Postgres
- Frontend never talks directly to Redis or Postgres
- Worker usually does not talk directly to the frontend

10. Frontend and Backend API Connection

The React frontend should call the Express API over HTTP.

Common frontend actions:

- Sign up / login
- Submit URL for audit
- Get audit history
- Get audit result by ID

Example API routes:

```
POST /api/auth/register
POST /api/auth/login
POST /api/audits
GET /api/audits
GET /api/audits/:id
```

Example request when a user clicks Run Audit:

```
POST /api/audits
Content-Type: application/json

{
  "url": "https://example.com"
}
```

Example fast backend response:

```
{
  "auditId": "123",
  "status": "pending"
}
```

The frontend can then redirect to the results page, poll for updates, and show a loading state.

11. Backend API to Postgres

The backend stores and reads application data from Postgres.

- Users
- Audit records
- Audit status
- Past results
- Metrics
- Suggestions

Example when the user submits an audit:

```
audits
- id: 123
- user_id: 45
- url: https://example.com
```

```
- status: pending
- created_at: ...
```

After creating that row, the backend sends the job to Redis. Postgres remains the source of truth for audit data.

12. Backend API to Redis Queue

The backend should not run Lighthouse directly. Instead, it should create a BullMQ job in Redis.

Example job payload:

```
{
  "auditId": 123,
  "url": "https://example.com",
  "userId": 45
}
```

The backend's job is to validate the URL, create the pending audit record in Postgres, enqueue the job in Redis, and return the audit ID quickly to the frontend.

13. Worker to Redis Queue

The worker listens for jobs from BullMQ.

Worker job flow:

- Receive job from queue
- Get auditId and URL
- Mark audit as running
- Launch Puppeteer and Lighthouse
- Parse results
- Save results to Postgres
- Mark audit as complete or failed

Redis is the bridge between the backend and the worker. The backend says, 'Here is work to do.' The worker says, 'I will take it.'

14. Worker to Postgres

After the worker finishes the audit, it writes the results into Postgres.

Example data written:

```
audits table
id: 123
status: complete
score: 84
completed_at: ...

metrics table
audit_id: 123
ttfb: 320
fcp: 1400
lcp: 2600
bundle_size: 510
```

```
image_weight: 1200

suggestions table
audit_id: 123
type: image
message: Compress large images
impact: high
```

Later, the backend can fetch that data and send it back to the frontend.

15. How the Frontend Gets Results Back

The frontend does not connect to the worker directly. It asks the backend for the report.

```
GET /api/audits/123
```

Example backend response:

```
{
  "id": 123,
  "status": "complete",
  "score": 84,
  "metrics": {
    "ttfb": 320,
    "fcp": 1400,
    "lcp": 2600
  },
  "suggestions": [
    {
      "type": "image",
      "message": "Compress large images",
      "impact": "high"
    }
  ]
}
```

The frontend then renders the dashboard.

16. End-to-End Flow

1. User submits a URL on the frontend.
2. Frontend sends POST /api/audits to the backend.
3. Backend validates the URL.
4. Backend creates a pending audit row in Postgres.
5. Backend pushes the audit job into Redis.
6. Worker consumes the job from Redis.
7. Worker runs the Lighthouse / Puppeteer audit.
8. Worker saves metrics, results, and suggestions into Postgres.
9. Frontend asks the backend for GET /api/audits/:id.
10. Backend reads from Postgres and returns the completed report.

17. Best Way to Structure the Code Connections

Frontend

Create an API service file:

```
client/src/services/auditService.js
```

Example functions:

- createAudit(url)
- getAuditById(id)
- getAuditHistory()

Backend

Split it into routes, controllers, services, queue logic, and DB logic.

```
server/src/routes/auditRoutes.js
server/src/controllers/auditController.js
server/src/services/auditService.js
server/src/queue/auditQueue.js
```

Worker

Split it into queue consumer, audit processor, Lighthouse runner, and suggestion generator.

```
worker/src/queues/auditWorker.js
worker/src/processors/processAudit.js
worker/src/services/lighthouseService.js
worker/src/services/suggestionService.js
```

18. Shared Data Between Backend and Worker

The backend and worker should agree on:

- Audit statuses
- Job payload format
- Database schema
- Suggestion types

Example shared constants:

```
export const AUDIT_STATUS = {
  PENDING: "pending",
  RUNNING: "running",
  COMPLETE: "complete",
  FAILED: "failed"
};
```

These can live in a shared folder inside the monorepo.

19. Real-Time Updates for MVP

For the MVP, the easiest option is polling rather than WebSockets.

The frontend can check every few seconds:

```
GET /api/audits/:id
```

It keeps doing that until the status becomes complete or failed.

- Submit audit
- Redirect to /audits/123
- Poll every 3 seconds

That is enough for the MVP and much simpler than adding socket infrastructure.

20. Environment Variables

Each service should have its own environment variables.

Frontend

```
VITE_API_URL=
```

Backend

```
PORT=
DATABASE_URL=
REDIS_URL=
JWT_SECRET=
CLIENT_URL=
```

Worker

```
DATABASE_URL=
REDIS_URL=
```

21. Simplest Mental Model

The frontend sends requests to the backend, the backend stores audit records in Postgres and queues jobs in Redis, the worker pulls jobs from Redis, runs Lighthouse audits, stores the results back in Postgres, and the frontend fetches those results from the backend.